RAPPORT DE PROJET

Ingénierie en Réseaux et Systèmes d'Information (IRISI)

Plateforme d'automatisation de workflows

Réalisé par :

Imane IRAFAN

Élève ingénieure en IRISI

Encadrant :

Prof. BENCHAREF

FST de Marrakech

Année universitaire 2026/2027

Note au lecteur

Ce rapport rassemble dans un seul fichier les informations de presentation, la documentation technique, le guide utilisateur, les instructions d'installation et le plan de validation de FlowForge. Il a ete redige a partir du code et des documents des depots `FlowForge-Frontend`, `FlowForge-Backend` et `FlowForge-ML`, consultes le 27 octobre 2026.

Les champs en rouge doivent etre personnalises avant remise. Les informations academiques, la methodologie formelle et les resultats experimentaux qui ne sont pas etablis par les depots ne sont pas inventes. Les validations historiques sont attribuees a leur compte rendu ; aucun test ni service n'a ete execute pendant la redaction de ce rapport. Les secrets et identifiants prives sont exclus.

Table des matières

Resume **4**

Abstract **5**

1 Introduction **6**
1.1 Contexte 6
1.2 Problematique 6
1.3 Objectifs 6
1.4 Perimetre et methode documentaire 7
1.5 Acteurs 7

2 Analyse fonctionnelle **8**
2.1 Besoins fonctionnels observes 8
2.2 Contraintes 8
2.3 Positionnement 8

3 Architecture et conception **9**
3.1 Vue generale 9
3.2 Technologies 9
3.3 Cycle d'execution 10
3.4 References de donnees 10
3.5 Modele de donnees 10

4 Documentation technique **12**
4.1 Organisation des depots 12
4.2 Frontend 12
4.3 API REST 12
4.4 Authentification et credentials 13
4.5 Catalogue des noeuds 13
4.6 Integrations et delais 14
4.7 Securite et limites 14

5 Service de machine learning **15**
5.1 Pipeline et donnees 15

5.2 Contrat HTTP 15

6 Guide utilisateur 16
6.1 Creer un workflow 16
6.2 Declencheurs 16
6.3 Connecter Google Sheets 16
6.4 Agent IA Ollama 17
6.5 Executer et diagnostiquer 17
6.6 Activation et historique 17

7 Guide d'installation locale 18
7.1 Prerequis 18
7.2 Backend 18
7.3 Frontend 19
7.4 Service ML 19
7.5 Ordre de verification local 19

8 Plan de tests et resultats disponibles 21
8.1 Tests declares dans les depots 21
8.2 Commandes disponibles 21
8.3 Matrice de verification recommandee 22
8.4 Validation historique rapportee 22
8.5 Donnees a consigner pour une campagne reelle 23

9 Limites et perspectives 24

10 Conclusion 25

A Configuration et commandes de reference 26

B References techniques 27

Resume

FlowForge est une application web qui permet de composer des workflows a partir de noeuds relies, de les enregistrer puis de les executer. Le projet est reparti en trois services : une interface React et TypeScript, une API Python/FastAPI chargee de l'authentification, de la persistance et de l'orchestration, et un service ML distinct exposant une demonstration de prediction tabulaire.

Le backend prend en charge des declencheurs manuels, webhook, planifies, Gmail et Telegram, ainsi que des actions de transformation, de controle conditionnel, HTTP, Google Sheets, messagerie, prediction ML et agent IA connecte a Ollama. Les executions sont placees dans Redis et traitees par un worker ARQ. PostgreSQL conserve les utilisateurs, workflows, credentials et journaux d'execution. Le moteur valide le graphe, ordonne les noeuds, resout les expressions entrantes et enregistre les etats et erreurs.

Ce document decrit les comportements visibles dans les depots. Il ne revendique ni disponibilite en production, ni conformite reglementaire, ni niveau de performance non mesure. Les validations citees sont des resultats historiques rapportes et ne signifient pas qu'elles ont ete rejouees pour ce rapport.

Mots-cles : automatisation, workflow, integration, FastAPI, React, agent IA, execution asynchrone.

Abstract

FlowForge is a web application for building and running workflows made of connected nodes. It consists of a React and TypeScript frontend, a FastAPI backend for authentication, persistence and orchestration, and a separate machine-learning service exposing a tabular prediction example. The backend supports manual, webhook, scheduled, Gmail and Telegram triggers, together with data transformation, conditional branching, HTTP requests, messaging, Google Sheets and an Ollama-backed AI agent. Redis and ARQ queue workflow executions, while PostgreSQL stores application data and execution records. This report documents implementation visible in the repositories; personal academic details and newly measured results must be completed by the student.

Chapitre 1

Introduction

1.1. Contexte

Les applications et services d'une organisation produisent des evenements et des donnees dans des systemes differents. Repeter manuellement les memes transferts ou traitements augmente le temps consacre aux taches operationnelles et rend les etapes difficiles a suivre. Un moteur de workflows offre un moyen visuel de decrire une chaine d'actions et de conserver une trace de son execution.

FlowForge explore cette approche dans une application web composee de services separes. Le depot montre un editeur graphique, une API d'orchestration et un service ML autonome. La presence d'integrations et d'un agent IA constitue un perimetre fonctionnel ; elle ne signifie pas que FlowForge reproduit toutes les capacites d'une plateforme existante.

1.2. Problematique

Comment proposer une application qui permet de construire visuellement un enchainement d'evenements et d'actions, de le lancer de maniere controlee et de consulter le resultat de chaque etape, tout en integrant des services externes ?

1.3. Objectifs

— Fournir une interface de creation et d'edition de graphes de workflows.

— Enregistrer les workflows et leurs parametres dans un backend authentifie.

— Valider les graphes et executer leurs noeuds dans un ordre coherent.

— Prendre en charge plusieurs modes de declenchement et des integrations externes.

— Afficher l'etat des executions et les erreurs par noeud.

— Isoler un service de prediction ML du backend principal.

1.4. Perimetre et methode documentaire

Le perimetre decrit correspond au code present dans les trois depots au moment de l'inspection. Il comprend l'editeur, les API de gestion, le moteur d'execution, les workers, les integrations codees et le service de demonstration ML. Les depots ne permettent pas de confirmer une methode de gestion de projet formelle. La methodologie suivie, les decisions de conception, le calendrier et les retours utilisateurs doivent etre ajoutes par l'etudiant a partir de ses propres traces.

1.5. Acteurs

Acteur	**Besoin ou role**
Utilisateur	Concevoir, lancer et suivre ses workflows ; gerer ses credentials.
Services externes	Emettre des evenements ou recevoir les appels du workflow.
Worker / administrateur	Executer les taches et configurer les services techniques.

Chapitre 2

Analyse fonctionnelle

2.1. Besoins fonctionnels observes

L'application propose l'inscription et la connexion, la gestion des workflows appartenant a l'utilisateur, la validation et l'execution d'un graphe, l'historique des executions et la gestion de credentials. Les types de noeuds disponibles dans le code sont enumeres au chapitre 4. Les interfaces de configuration ne sont pas toutes specifiques : plusieurs noeuds utilisent encore une configuration JSON generique.

2.2. Contraintes

— Le backend depend de PostgreSQL, Redis et d'un worker ARQ pour le traitement asynchrone.

— Les graphes et plusieurs resultats sont stockes en JSONB, liant le schema a PostgreSQL.

— Le service de prediction ML est distinct et appele par HTTP.

— Les integrations tierces necessitent leurs propres comptes, scopes, droits et secrets.

— Gmail et Telegram sont interroges periodiquement par le worker ; ce fonctionnement ne garantit pas une reception instantanee.

2.3. Positionnement

FlowForge suit le principe general d'outils d'automatisation par workflows : associer des declencheurs et des actions dans un editeur visuel. Le code consulte ne permet pas une comparaison complete ou mesuree avec n8n ou une autre plateforme. FlowForge est presente ici comme un projet academique d'automatisation inspire par ce type d'interface, sans revendication de parite fonctionnelle.

Chapitre 3

Architecture et conception

3.1. Vue generale

Le frontend appelle l'API REST FastAPI. L'API authentifie les requetes, verifie la propriete des ressources et utilise PostgreSQL. Une demande d'execution est placee dans Redis ; le worker ARQ charge le graphe, execute les noeuds et persiste chaque etat. Les workers surveillent egalement les declencheurs planifies et les integrations de polling. L'agent IA appelle l'API Ollama configuree. Le noeud ML appelle le service FlowForge-ML.

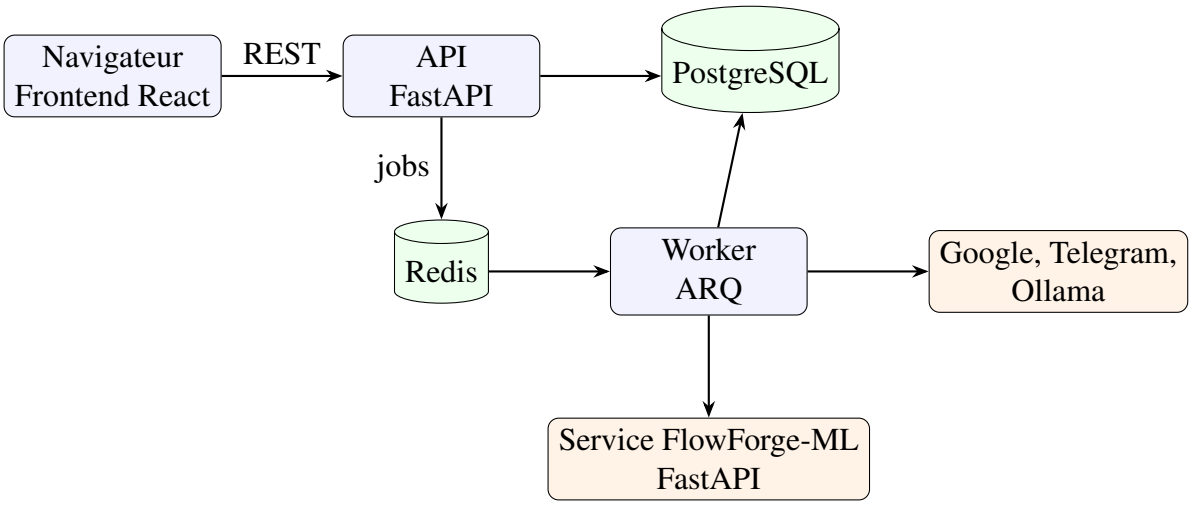


FIGURE 3.1 – Architecture logique des services FlowForge.

3.2. Technologies

Couche	Technologies	Role
Interface	React 19, TypeScript, Vite, React Router, @xyflow/react, Tailwind CSS	Editeur graphique et vues de gestion.

API	Python, FastAPI, Pydantic, SQLAlchemy, Alembic	API REST, validation, acces aux donnees et migrations.
Execution asynchrone	Redis, ARQ	Mise en file et traitement des executions.
Persistance	PostgreSQL	Utilisateurs, workflows, credentials et historique.
IA generative	API chat Ollama	Generation de texte ou sortie structuree depuis un noeud IA.
Service ML	FastAPI, scikit-learn, pandas, joblib, MLflow optionnel	Entrainement, suivi facultatif et inference.
Qualite	Pytest, Vitest, scripts typecheck et lint	Tests et controles de code declares par les depots.

3.3. Cycle d'execution

Le backend valide le graphe (identifiants, liens, types et absence de cycle), puis l'ordonne topologiquement. L'API place l'execution dans Redis via ARQ ; le worker traite les noeuds, appelle les services requis et persiste statuts, sorties et erreurs dans PostgreSQL. Le frontend interroge l'API pour afficher le suivi. Les branches non retenues et les noeuds bloques en aval sont marques comme ignores.

3.4. References de donnees

Les templates utilisent l'identifiant technique d'un noeud et un chemin de sortie, par exemple `{{ node-id.body.value }}`. L'exemple doit etre remplace par l'ID reel. Une entree de workflow peut etre referencee sous `input_data`. Le moteur resout une reference a partir d'un resultat deja produit. Les syntaxes n8n telles que `$json` ou `$('Nom du noeud')` ne sont pas des alias documentes par ce moteur. Une reference inexistante ou vers un noeud non execute genere une erreur.

3.5. Modele de donnees

Entite	**Donnees principales**
users	UUID, email unique, hash du mot de passe, nom, etat actif et horodatages.

workflows	UUID, proprietaire, nom, description, graphe JSONB, etat actif.
triggers	Workflow, type, configuration JSONB, token/secret webhook, calendrier et etat.
workflow_executions	Workflow, utilisateur, etat, type de declenchement, cle d'evenement, entree, erreur et horodatages.
node_executions	Execution, ID/type du noeud, etat, sortie JSONB, erreur et duree.
credentials	Proprietaire, nom/type et donnees chiffrees.
oauth_states	Utilisateur, nonce unique et expiration.

Les modeles et migrations emploient des types PostgreSQL tels que JSONB, UUID et enums. La migration la plus recente inspectee ne fournit pas de downgrade implemente ; un retour arriere doit etre planifie avec prudence.

Chapitre 4

Documentation technique

4.1. Organisation des depots

Depot	Responsabilite
FlowForge-Frontend	Application React/TypeScript, editeur, navigation et client API.
FlowForge-Backend	API, modeles, migrations, moteur, integrations et workers.
FlowForge-ML	API de prediction, commande d'entrainement et suivi MLflow facultatif.

4.2. Frontend

Le client API lit `VITE_API_BASE_URL` ; l'exemple pointe vers `http://localhost:8000/api/v1`. Le frontend utilise React Router pour la navigation et @xyflow/react pour l'editeur en graphe. Les scripts declares comprennent dev, build, preview, typecheck, lint, test et test :watch. Le README recommande Node 22.12 ou plus recent. Le jeton de session est garde en memoire dans le contexte React et non dans localStorage ; une nouvelle session peut demander une reconnexion.

L'editeur propose le catalogue de noeuds. Des formulaires specifiques existent pour plusieurs configurations, notamment schedule, webhook, Gmail, agent IA et Google Sheets ; d'autres noeuds utilisent une configuration JSON generique. Pendant une execution, l'editeur interroge periodiquement l'API pour actualiser les etats.

4.3. API REST

Toutes les routes suivantes sont sous le prefixe `/api/v1`. Les routes de ressources exigent un jeton bearer et un controle de propriete, sauf routes publiques de sante ou webhook.

Methode et route	Fonction
`/auth/register`, `/auth/login`	Inscription et authentification JWT.
`/workflows` et `/workflows/{id}`	Creer, lister, lire, modifier, supprimer et activer un workflow.
`/workflows/{id}/executions`, `/executions`	Lancer et consulter les executions ; annulation disponible.
`/webhooks/{token}`	Recevoir un evenement JSON pour un workflow actif.
`/credentials`	Gerer les credentials ; les secrets ne sont pas retournes en lecture.
`/integrations/google/*`	Demarrer OAuth Google et recevoir son callback.
`/health`, `/docs`	Sante et documentation OpenAPI.

Les pages FastAPI `/docs`, `/redoc` et `/openapi.json` exposent le schema de l'API. Le schema OpenAPI de la version lancee est la reference pour les corps et reponses exacts.

4.4. Authentification et credentials

L'authentification emploie un JWT bearer avec une expiration configurable (30 minutes par defaut dans les parametres inspectes). Les mots de passe sont haches. Les credentials sont chiffres avant persistance ; les routes de lecture retournent les metadonnees, pas les valeurs secretes. Types de credentials identifies : api_key, oauth2_google, telegram_bot et http_header.

Le flux Google associe un etat signe a un nonce a usage unique persiste et expire apres dix minutes. Les scopes codes couvrent Gmail en lecture/envoi et Google Sheets. Le compte OAuth doit avoir le droit editeur sur la feuille. Une cle API Google seule n'autorise pas l'ecriture dans une feuille privee. Apres un changement des scopes demandes, une nouvelle autorisation peut etre necessaire.

4.5. Catalogue des noeuds

Le catalogue comprend cinq declencheurs (manuel, webhook, planifie, Gmail, Telegram), des noeuds de transformation et condition, des actions HTTP/Gmail/Telegram/Google Sheets, un agent IA Ollama et une prediction FlowForge-ML. Leur usage exige la configuration des credentials et du service externe correspondant.

4.6. Integrations et delais

— Gmail et Telegram sont detectes par polling. L'intervalle observe pour Telegram est configurable de 1 a 3600 secondes, avec une valeur par defaut recente d'une seconde ; Gmail a un minimum de 30 secondes et une valeur par defaut de 60 secondes. Ces intervalles ne garantissent pas la latence bout en bout.

— Google Sheets ajoute une ligne dans une plage A1 et requiert OAuth avec le scope Sheets et un acces editeur.

— Ollama recoit une requete chat au modele configure. Le delai depend du reseau, de la charge et du modele.

— Le service FlowForge-ML est appele par HTTP a l'URL configuree.

— Un webhook peut verifier `X-FlowForge-Signature` au format `sha256=<HMAC-SHA256 du corps>` lorsqu'un secret est configure.

4.7. Securite et limites

Les controles visibles comprennent jetons bearer, hachage des mots de passe, chiffrement des credentials, verification de propriete et validation de destinations HTTP pour reduire les requetes vers des adresses privees/reservees. Ces mecanismes ne constituent pas une certification ni une revue de securite globale. Un deploiement reel exige notamment gestion des secrets, configuration TLS, mises a jour, supervision et tests de securite.

Chapitre 5

Service de machine learning

5.1. Pipeline et donnees

Le service ML fournit une demonstration de classification du jeu Wisconsin Breast Cancer de scikit-learn : 30 caracteristiques numeriques, separation stratifiee 80/20 avec `random_state=42`, `StandardScaler`, puis `LogisticRegression` avec `max_iter=2000`. Les labels sont 0 = malignant et 1 = benign.

Le modele joblib et le rapport d'entrainement sont enregistres dans le dossier d'artefacts configure ; par defaut, le modele est `artifacts/breast_cancer_logistic_regression.joblib`. MLflow est facultatif et separe de l'API d'inference. Cette demonstration est pedagogique et ne doit pas servir au diagnostic medical.

5.2. Contrat HTTP

L'API ecoute par defaut sur `127.0.0.1:8001`.

Route	Fonction
`GET /health`	Indiquer l'etat et si le modele est pret.
`GET /models`	Lister le modele local charge.
`POST /predict`	Predire avec un model_id et les 30 caracteristiques attendues.

Les champs absents, supplementaires ou invalides entrainent une erreur 422 ; modele non pret 503 ; ID inconnu 404 ; erreur d'inference 500. La reponse inclut prediction, confiance, probabilites, ID et duree.

Chapitre 6

Guide utilisateur

6.1. Creer un workflow

1. Se connecter et creer un workflow depuis la liste.
2. Ajouter un declencheur et des actions, puis relier les noeuds.
3. Configurer les champs, les expressions entrantes et les credentials.
4. Enregistrer et commencer par une execution manuelle de test.

Le nom visible et l'ID technique d'un noeud sont distincts. Les expressions utilisent l'ID technique. Inspecter la sortie du noeud precedent pour determiner le chemin de propriete reel.

6.2. Declencheurs

— Manuel : lancer depuis l'interface et fournir les donnees JSON. Les donnees d'exemple simulent un evenement, sans activer d'ecoute permanente.

— Webhook : activer le workflow et envoyer un JSON a l'URL fournie ; une signature HMAC peut etre exigee.

— Schedule : definir cron et fuseau horaire, puis activer ; le worker doit rester disponible.

— Gmail/Telegram : connecter le compte ou bot et activer. Le delai depend du polling et du worker.

6.3. Connecter Google Sheets

1. Connecter le compte Google OAuth et accepter le scope Sheets.
2. Reconnecter le compte si le scope a ete ajoute apres une ancienne autorisation.
3. Dans le noeud, choisir le credential, l'ID spreadsheet, la plage A1 et les valeurs.
4. Verifier que l'API est active et que le compte est editeur de la feuille.

Une cle API seule ne donne pas acces en ecriture a une feuille privee. Pour un 403 de scope, renouveler l'autorisation ; pour un 400 de plage, verifier l'ID, l'onglet et la notation A1.

6.4. Agent IA Ollama

Configurer l'acces Ollama dans l'environnement backend, choisir le modele et remplir le prompt utilisateur ; les instructions systeme, temperature, limite de tokens et format peuvent etre configures selon le formulaire. Pour transmettre une valeur, referencer un champ present dans la sortie d'un noeud precedent. La latence depend du modele et de son hebergement.

6.5. Executer et diagnostiquer

Enregistrer, lancer avec des donnees de test non sensibles, puis ouvrir tout noeud echoue pour lire son message et ses donnees avant de corriger.

Symptome	**Verification**
Unresolved template reference	ID source correct, noeud connecte et execute, chemin present dans sa sortie ?
key not found	Le nom de propriete correspond-il au JSON retourne ?
403 Google	Le compte a-t-il acces a la feuille et le jeton possede-t-il le scope Sheets ?
400 Google Sheets range	L'ID, le nom d'onglet et la plage A1 sont-ils corrects ?
Execution reste queued	Redis et le worker sont-ils demarres et utilisent-ils la meme configuration ?
Agent IA lent	Verifier modele, reseau, charge et longueur du prompt/reponse.

Etats d'execution : queued, running, success, failed, cancelled. Etats de noeud : pending, running, success, failed, skipped. Un noeud skipped n'a pas ete execute sur ce parcours, par exemple apres un echec amont ou une autre branche.

6.6. Activation et historique

Un lancement manuel ponctuel n'est pas une ecoute permanente. Pour les declencheurs planifies, Gmail, Telegram ou webhook, configurer et activer le workflow et maintenir les workers disponibles. Consulter ensuite l'historique ; actualiser le detail si necessaire. Les sorties peuvent contenir des donnees metier et doivent etre anonymisees avant partage.

Chapitre 7

Guide d'installation locale

7.1. Prerequis

— Git, PowerShell et Docker Desktop avec Docker Compose pour les dependances conteneurisees.

— Python 3.11 recommande pour le backend ; le service ML exige Python 3.10 ou plus recent.

— Node.js 22.12 ou plus recent recommande par le README frontend.

— Un navigateur ; comptes/API tiers seulement pour les integrations employees.

Ce guide vise un environnement local de developpement, pas un deploiement production. Ne pas versionner les fichiers `.env` ; les valeurs compose de developpement ne doivent pas etre exposees publiquement.

7.2. Backend

Depuis le repertoire `FlowForge-Backend`, creer et configurer un fichier d'environnement local :

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
```

Configurer les URL PostgreSQL/Redis, une cle JWT et une cle de chiffrement generees localement. Completer les parametres d'integration uniquement si necessaires. Puis lancer les dependances :

```
docker compose up -d db redis
```

Apres installation Python et configuration :

```
alembic upgrade head
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Dans un second terminal, demarrer le worker :

```
.\.venv\Scripts\Activate.ps1
python -m app.workers.main
```

Mode conteneurise alternatif :

```
docker compose up --build -d db redis
docker compose run --rm api alembic upgrade head
docker compose up --build -d api worker
```

Verifier les logs, `/health` et `/docs`. L'API seule ne consomme pas les jobs ARQ. Le Dockerfile definit un healthcheck HTTP vers l'API ; le processus worker n'expose pas necessairement ce port, donc consulter ses logs pour verifier son activite.

7.3. Frontend

Depuis `FlowForge-Frontend` :

```
npm install
Copy-Item .env.example .env.local
npm run dev
```

Verifier `VITE_API_BASE_URL` selon l'adresse effective de l'API, puis ouvrir l'URL indiquee par Vite.

7.4. Service ML

Depuis `FlowForge-ML` :

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
flowforge-train --output-dir artifacts --no-tracking
flowforge-api
```

Verifier `GET /health` sur le port 8001. Configurer `FLOWFORGE_ML_URL` du backend vers une adresse accessible depuis le processus backend. En conteneur, 127.0.0.1 designe le conteneur courant et non un autre service.

7.5. Ordre de verification local

1. Controler PostgreSQL et Redis.
2. Verifier l'API avec `/health` et `/docs`.

3. Verifier les logs du worker.
4. Verifier l'API ML si le noeud prediction est utilise.
5. Lancer le frontend, s'inscrire, puis executer un workflow manuel simple.
6. Ajouter les integrations une par une.

Ces instructions n'ont pas ete executees pendant la redaction. Pour arreter les conteneurs sans supprimer les volumes :

```
docker compose stop
```

Les terminaux Python/Node peuvent etre arretes avec Ctrl+C. Le compose comprend une base de test distincte ; lire les gardes de tests avant toute commande qui pourrait toucher une base. Ne pas diriger des tests vers une base contenant des donnees utiles.

Chapitre 8

Plan de tests et resultats disponibles

8.1. Tests declares dans les depots

Composant	Couverture et outils observes
Backend	Pytest, pytest-asyncio et pytest-cov ; tests unitaires et d'integration. La configuration utilise SQLite pour les tests unitaires et une base PostgreSQL isolee pour l'integration.
Frontend	Vitest ; validation de graphe, client API, workflows/credentials/palette et panneau de configuration. Scripts de typecheck et lint.
FlowForge-ML	Pytest ; tests d'entrainement, tracking et API.

8.2. Commandes disponibles

Ces commandes sont repertoriees dans les projets ; elles n'ont pas ete executees pour ce rapport.

```
# Backend
python -m pytest
python -m pytest tests/unit/
python -m pytest tests/integration/
python -m pytest --cov=app --cov-report=html

# Frontend
npm test
npm run typecheck
npm run lint

# FlowForge-ML
python -m pytest
```

Les tests d'integration backend necessitent une base PostgreSQL de test jetable configuree. Lire les gardes de `tests/conftest.py` ; ne pas employer une base de donnees contenant des informations utiles.

8.3. Matrice de verification recommandee

ID	Scenario	Resultat attendu
AUTH	Inscription, connexion et controle de propriete.	Acces attendu, sans fuite de donnees.
GRAPH	Graphe valide, cycle ou reference invalide.	Graphe valide accepte ; erreur invalide expliquee.
EXEC	Chemin manuel, branche, echec ou annulation.	Statuts et sorties coherents par noeud.
TRIGGER	Webhook, schedule, Gmail et Telegram.	Evenement traite selon configuration et polling.
SHEETS	Append avec scope correct ou permission invalide.	Ligne ajoutee ou erreur explicite.
AI/ML	Requete Ollama et prediction valide/invalide.	Sortie conforme ou erreur de validation.
SECURITY	Secret de credential et destination HTTP reservee.	Secret masque ; destination interdite refusee.
OPERATIONS	API disponible avec worker absent.	Attente ou echec diagnostiquable, sans faux succes.

Cette matrice est un plan propose et ne garantit pas que chaque cas dispose deja d'un test automatise.

8.4. Validation historique rapportee

Le fichier `FlowForge-Frontend/docs/INTEGRATION_VALIDATION.md`, date du 19 octobre 2026, rapporte un parcours navigateur/Playwright couvrant inscription/connexion, dashboard, creation d'un graphe, sauvegarde/rechargement, activation, execution et historique. Il rapporte aussi une validation du cycle schedule et le resultat `2 passed` pour le test d'integration correspondant.

Ces resultats proviennent d'un compte rendu existant et n'ont pas ete rejoues lors de la redaction. Les ports, versions et conditions de ce compte rendu doivent etre revérifiés avant de les presenter comme reproductibles.

8.5. Donnees a consigner pour une campagne reelle

— Commit/version exacte, systeme, versions Python/Node et navigateur.

— Services et URL utilises, sans secrets.

— Commandes executees, statut final et logs expurges.

— Cas reussis/echoues, anomalies et etapes de reproduction.

— Mesures de duree avec repetitions pour toute affirmation de performance.

— Jeu de donnees et metriques effectivement utilises pour evaluer le modele.

Chapitre 9

Limites et perspectives

— Les triggers de polling ont un delai dependant de leur intervalle, du worker et du fournisseur.

— Les interfaces de configuration de noeuds ne sont pas toutes guidees par des champs specifiques.

— Retries, limites de debit et reprise apres panne doivent etre evalues integration par integration.

— Les journaux d'execution ne remplacent pas une observabilite centralisee ni des alertes.

— Le modele ML fourni est une demonstration educative, sans usage clinique.

— Les essais de charge, de securite, de compatibilite et d'experience utilisateur restent a completer.

— Une comparaison de solutions doit s'appuyer sur des sources datees et des criteres explicites.

Chapitre 10

Conclusion

FlowForge rassemble une interface visuelle de workflow, un moteur backend asynchrone, plusieurs connecteurs et un service ML independant. Le code met en place une chaine allant de la construction et validation du graphe a la mise en file, l'execution et le suivi par noeud. Les validations deja consignees forment un point de depart, mais elles ne remplacent pas une campagne actuelle de tests ni les informations academiques propres a l'etudiant.

Avant remise, completer la page de garde, la methodologie effectivement suivie, les captures d'ecran expurgees, les resultats de tests rejoues et toute mesure ou comparaison reellement effectuee. Ne pas presenter comme resultat acquis une fonctionnalite ou une mesure qui n'a pas ete verifiee.

Annexe A

Configuration et commandes de reference

Les variables de configuration principales sont :

Variable	Utilite
DATABASE_URL	Connexion SQLAlchemy a PostgreSQL.
REDIS_URL	Connexion Redis pour ARQ.
SECRET_KEY, ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES	Signature et duree des jetons.
ENCRYPTION_KEY	Chiffrement des credentials.
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI	OAuth Google.
FRONTEND_URL	Origine frontend pour les retours d'integration.
TELEGRAM_BOT_TOKEN	Parametre disponible au connecteur Telegram.
FLOWFORGE_ML_URL	URL du service de prediction.
OLLAMA_API_KEY, OLLAMA_BASE_URL, OLLAMA_MODEL	Acces et modele de l'agent IA.
LOG_LEVEL, WORKERS_COUNT, ENVIRONMENT, APP_VERSION	Journalisation et fonctionnement.

Les valeurs sont volontairement omises. Utiliser les fichiers `.env.example` comme point de depart et les parametres du code comme reference des valeurs effectives. Ne jamais publier les secrets.

Annexe B

References techniques

Bibliographie

[1] FastAPI, *Tutorial*, https://fastapi.tiangolo.com/tutorial/.

[2] Pydantic, *Documentation*, https://docs.pydantic.dev/latest/.

[3] SQLAlchemy, *ORM Documentation*, https://docs.sqlalchemy.org/en/20/orm/.

[4] Alembic, *Tutorial*, https://alembic.sqlalchemy.org/en/latest/tutorial.html.

[5] Redis, *Documentation*, https://redis.io/docs/latest/.

[6] ARQ, *Worker documentation*, https://arq-docs.helpmanual.io/.

[7] React, *Learn React*, https://react.dev/learn.

[8] Vite, *Guide*, https://vite.dev/guide/.

[9] React Flow, *Learn*, https://reactflow.dev/learn.

[10] Docker, *Compose documentation*, https://docs.docker.com/compose/.

[11] Google, *Gmail API : List messages*, https://developers.google.com/workspace/gmail/api/guides/list-messages.

[12] Google, *Sheets API : Append values*, https://developers.google.com/workspace/sheets/api/reference/rest/v4/spreadsheets.values/append.

[13] Google, *OAuth 2.0 for server-side web apps*, https://developers.google.com/identity/protocols/oauth2/web-server.

[14] Telegram, *Bot API*, https://core.telegram.org/bots/api.

[15] Ollama, *Chat API*, https://docs.ollama.com/api/chat.

[16] Scikit-learn, *Model persistence*, https://scikit-learn.org/stable/model_persistence.html.

[17] MLflow, *ML Tracking*, https://mlflow.org/docs/latest/ml/tracking/.